A Project Report

on

Evaluation of Methodologies for Improving and Enhancing LLM Agents for General OS Tasks

by

1. Aditya Ahire
2. Aniruddha Chaudhari
3. Raj Malave
4. Soham Mahale

under the guidance of

Prof. Savita Lade

MANJARA CHARITABLE TRUST

RAJIV GANDHI INSTITUTE OF TECHNOLOGY, MUMBAI

Juhu-Versova Link Road Versova, Andheri(W), Mumbai-53.

Department of Computer Engineering

University of Mumbai

April - 2026

MANJARA CHARITABLE TRUST

RAJIV GANDHI INSTITUTE OF TECHNOLOGY, MUMBAI

Juhu-Versova Link Road Versova, Andheri(W), Mumbai-53.

Department of Computer Engineering

Certificate

This is to certify that

1. Aditya Ahire
2. Aniruddha Chaudhari
3. Raj Malave
4. Soham Mahale

Have satisfactory completed this project entitled

Evaluation of Methodologies for Improving and Enhancing LLM Agents for General OS Tasks

Towards the partial fulfillment of the

BACHELOR OF ENGINEERING
IN
(COMPUTER ENGINEERING)

as laid by University of Mumbai.

Prof. Savita Lade	**Prof.S. P. Khachane**
Guide	**Head of Department**

Principal

Dr.Sanjay Bokade

Project Report Approval for B. E.

This project report entitled **"Evaluation of Methodologies for Improving and Enhancing LLM Agents for General OS Tasks"** by **Aditya Ahire, Anirudha Chaudhary, Raj Malave, Soham Mahale** is approved for the degree of Computer Engineering.

Examiners

1. ——————————

2. ——————————

Date:

Place:

Declaration

We wish to state that the work embodied in this project titled ”Evaluation of Methodologies for Improving and Enhancing LLM Agents for General OS Tasks” forms our own contribution to the work carried out under the guidance of ”Prof. Savita Lade” at the Rajiv Gandhi Institute of Technology.

We declare that this written submission represents our ideas in our own words and where others’ ideas or words have been included, we have adequately cited and referenced the original sources. we also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission.

we understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which

have thus not been properly cited or from whom proper permission has not been taken when needed.

(Students Signatures)

Aditya Ahire (801)

Aniruddha Chaudhari(808)

Raj Malave (851)

Soham Mahale (850)

Abstract

Large language model (LLM) agents are increasingly used to perform general operating system (OS) tasks such as file management, command execution, log inspection, software configuration, and workflow automation. Despite rapid progress, performance remains in- consistent across environments due to tool-use errors, weak planning under uncertainty, and limited robustness to noisy system states. This paper evaluates key methodologies for im- proving LLM-agent reliability and effectiveness in general OS settings. We review prior work on agent architectures, tool augmentation, memory mechanisms, and feedback-driven optimization, then organize these methods into a unified evaluation framework. The frame- work compares approaches across task success rate, execution efficiency, safety, recovery from failure, and adaptability to unseen tasks. We further discuss practical trade-offs among prompting, fine-tuning, retrieval augmentation, verifier-based control, and multi-agent co- ordination. Results from comparative analysis highlight that hybrid strategies, combining structured planning, constrained tool interfaces, and iterative self-correction, consistently outperform single-technique baselines.

Contents

List of Algorithms iv

List of Symbols v

1 Introduction 1
1.1 Introduction Description 1
1.2 Organization of Report 1

2 Literature Review 3
2.1 Survey Existing system 3

3 Proposed System 4
3.1 Overview 4
3.2 Core Methodological Foundations 4
3.2.1 Model-Level Methodologies 4
3.2.2 Model Pool Used in This Work 5
3.2.3 Prompt-Level Design 5
3.2.4 Harness-Level Architecture 5
3.2.5 Context, Skill, and Memory Integration 5
3.3 Key Methodologies 5
3.3.1 Tool-Augmented Agent Execution (Grounded Action Loop) 5
3.3.2 Retrieval-Augmented Planning with QMD (Memory + Constraint Reuse) 6
3.3.3 Skill-Based Tooling (Office/Document Format Contracts) 6
3.3.4 Evaluator-Guided Self-Correction (Keyword + LLM Judge) 7
3.3.5 Better Verification for Evidence-Sensitive Tasks (Addressing Fabrication) 7
3.4 System Architecture (Textual) 7
3.5 Expected Benefits 8
3.6 Limitations of the Current Evaluation 9
3.7 Improvement Roadmap for Future Iterations 9

4 Experimental Setup 10
4.1 Agent Architecture and Harness Design 10
4.2 Model Configuration for Evaluation 10
4.3 Execution Flow Across Layers 11
4.4 Evaluation 11
4.4.1 Web Browsing 11
4.4.2 File-management 11

4.4.3 Google Workspace 12

5 Results & Discussions 13
5.1 Evaluation Scope and Key Definitions 13
5.2 Model-Level Effectiveness (file-management-skills) 13
5.2.1 Interpretation: Effectiveness vs Efficiency 14
5.3 Per-Task Results (Gemini Flash Example) 14
5.4 Qualitative Discussion: Dominant Failure Modes 15
5.4.1 Evidence-sensitive tasks 15
5.4.2 Formatting contracts and style constraints 15
5.4.3 Spreadsheet verification requirements 16
5.5 Efficiency, Thrashing, Safety, and Constraints (Gemini Flash) 16
5.6 Cross-Suite Comparison (Best Models by Success Rate) 16
5.7 Recommendations for Methodological Improvement 16

6 Conclusion & Future Work 20
6.1 Conclusion (What the Evaluation Shows) 20
6.1.1 Effectiveness is strongly model-dependent (file-management-skills, 14 tasks) 20
6.1.2 The dominant failure driver is execution and verification, not safety . 20
6.2 Key Takeaways (Methodological Findings) 21
6.3 Future Work (Concrete Next Steps) 21
6.3.1 Evidence-Binding Verification (OCR, Redline, and Table Extraction) 21
6.3.2 Contract-Based Tool Outputs (Schema + Repair) 21
6.3.3 Multi-Stage Plan -> Execute -> Verify -> Fix Loop 22
6.3.4 Ablation Studies Across Methodological Components 22
6.3.5 Evaluation Expansion 22
6.4 Final Summary 22

Appendix 23
Appendix A: Extended Evaluation Notes 23
Appendix B: Extended Rubric Guidance 23
B.1 Deterministic Checks 23
B.2 Judge-Assisted Semantic Checks 23
B.3 Recommended Decision Hierarchy 24
Appendix C: Error Taxonomy and Triage Procedure 24
C.1 Triage Steps 24
C.2 Category Interpretation Notes 24
Appendix D: Artifact Validation Checklists 25
D.1 Spreadsheet Tasks 25
D.2 Document Redline Tasks 25
D.3 OCR and Extraction Tasks 25
D.4 Presentation Tasks 25
Appendix E: Reproducibility Checklist 26
Appendix F: Ablation Plan Template 26
Appendix G: Additional Discussion Notes 26
Appendix H: Reviewer Checklist for Final Report Validation 28

References **29**

List of Algorithms

1. **Task Interpretation and Planning Algorithm**
   Parses task objectives, identifies constraints, and creates a structured execution plan.
2. **Tool Routing and Execution Algorithm**
   Selects the correct tool or skill module for each planned action and executes commands on real artifacts.
3. **Retrieval-Augmented Planning (QMD) Algorithm**
   Retrieves relevant prior runs, constraints, and guidance signals to improve planning quality.
4. **Contract Validation Algorithm**
   Checks generated outputs against required schema, formatting, and naming contracts.
5. **Evidence Binding Verification Algorithm**
   Verifies that extracted values are grounded in tool-produced evidence from source artifacts.
6. **Evaluator-Guided Repair Algorithm**
   Converts deterministic and judge feedback into targeted corrective actions.
7. **Iterative Control Loop Algorithm**
   Implements the workflow: Plan → Execute → Verify → Fix.
8. **Efficiency and Thrashing Analysis Algorithm**
   Computes trajectory-level metrics such as steps, unnecessary actions, tool calls, and thrash ratio.

List of Symbols

S	Final combined task score (0–100).
S_k	Deterministic keyword/format score (keywordScore).
S_j	Judge-based semantic score (judgeScore).
T	Total number of tasks in a suite.
P	Number of passed tasks (status == pass).
R_s	Success rate, $R_s = \frac{P}{T} \times 100$.
L_t	Trajectory length (number of execution steps for a task).
C_t	Total tool calls in a task trajectory.
U_t	Number of unnecessary steps for a task.
I_r	Inefficiency ratio for a task/run.
E_t	Elapsed time per task (seconds).
ϑ_{pass}	Pass threshold on score (typically ≥ 80).
$\vartheta_{partial}$	Partial threshold range (typically $30 \leq S < 80$).
ρ_{thrash}	Thrash ratio (degree of repetitive or non-productive action).
μ_{safe}	Mean safety score across evaluated tasks.
V_c	Constraint violation count.

Chapter 1

Introduction

1.1 Introduction Description

Large language model (LLM) agents are rapidly moving from conversational assistants to action-oriented systems that can execute real operating system (OS) tasks. In this setting, an agent must map high-level user intent (for example, set up a Python environment, run tests, and summarize failures) into a sequence of concrete interactions with file systems, shells, package managers, integrated development tools, and graphical interfaces. This shift from text-only generation to environment-coupled action introduces a harder reliability problem: a single incorrect command, invalid tool argument, or misread system state can propagate through the workflow and invalidate downstream decisions.

General OS task automation is challenging because system environments are dynamic, partially observable, and heterogeneous. The same instruction can require different action traces across Linux distributions, shell configurations, dependency versions, permission mod- els, and runtime constraints. Unlike static question-answering tasks, OS tasks are stateful: each action changes the environment and therefore changes what future actions are valid. Consequently, language reasoning alone is insufficient; successful agents require robust con- trol loops that couple planning, memory, tool invocation, verification, and recovery under uncertainty.

Recent advances in agent engineering suggest that performance gains are increasingly driven by system-design choices rather than model scale alone. Techniques such as con- strained function calling, hierarchical memory, verifier-guided correction, and multi-agent role decomposition can materially improve action validity and recovery rates. However, ex- isting literature often evaluates these techniques in isolation, with inconsistent metrics and benchmark scopes. This fragmentation makes it difficult to determine which methodologies generalize across realistic OS workloads and which improvements are benchmark specific.

1.2 Organization of Report

The remainder of this report is organized as follows:

- **Ch.1 Introduction:** Problem framing, reliability challenges in OS-agent execution, and motivation for methodology-driven evaluation.
- **Ch.2 Review of Literature:** Review of major research themes in agent evolution, modular skills, recursive inference, and architectural taxonomies.

- **Ch.3 Proposed System:** Model-level, prompt-level, and harness-level methodologies with context management, skill abstraction, and memory integration.
- **Ch.4 Experimental Setup:** Layered agent architecture and harness design used for evaluation and comparative analysis.
- **Ch.5 Results & Discussions:** Comparative findings across Web Browsing, File-management, and Google Workspace tasks with reliability, efficiency, safety, and robustness analysis.
- **Ch.6 Conclusion & Future Work:** Key conclusions, practical recommendations, and future extensions for robust general-task agent systems.

Chapter 2

Literature Review

2.1 Survey Existing system

The development of EvoCUA (Xue et al., 2026) marks a significant shift from static imitation learning to a self-sustaining evolutionary cycle for computer-use agents. By implementing a verifiable synthesis engine, this methodology enables autonomous generation of diverse OS tasks and their corresponding executable validators. The approach addresses the data-scarcity bottleneck by orchestrating large-scale asynchronous sandbox rollouts in QEMU-KVM environments. Its iterative evolving-learning strategy enables agents to internalize interaction experience rather than textual proxies, producing strong performance on OSWorld-style benchmarks.

The transition from monolithic architectures to modular systems is formalized in research on Agent Skills (Xu and Yan, 2026). This methodology advocates SKILL.md as a portable specification that allows agents to load composable packages of instructions and code on demand. Through integration with the Model Context Protocol (MCP), agents can decouple procedural knowledge from model weights and dynamically extend capabilities without retraining. This modularity is especially effective for OS tasks where tool-use protocols can be packaged as discrete skills, although security risks in community-contributed skill libraries remain an open concern.

To address performance degradation in long-horizon tasks, often referred to as context rot, Recursive Language Models (Zhang et al., 2026) introduce an inference-time scaling methodology. The approach treats long prompts and system state as an external environment that the agent can inspect and decompose programmatically. By recursively calling the model over smaller context segments through a Python REPL, the agent can process inputs far beyond its native context window. This methodology is especially relevant for OS tasks requiring state continuity across long action sequences.

A comprehensive architectural foundation is established in the survey of AI agent systems (Xu, 2026), which synthesizes reasoning, planning, and tool-calling components into a unified taxonomy spanning policy cores, world models, and tool routers. For OS-specific deployment, this framework highlights critical trade-offs between latency and decision accuracy, as well as non-determinism in long-horizon credit assignment. It also motivates evaluation metrics for measuring how reliably an agent translates natural-language intent into executable computation within real system environments.

Chapter 3

Proposed System

3.1 Overview

This project evaluates methodologies for improving LLM agents that must complete general-purpose tasks end-to-end, including planning, interacting with tools, producing artifacts, and self-correcting when failures occur.

Based on the current evaluation framework, the proposed system couples four core components:

1. A tool-augmented agent-browser execution loop for grounded actions on files and environments.
2. A domain/tool-skill layer for structured outputs across common office and document formats (for example, spreadsheets and reports).
3. A hybrid memory/retrieval component (QMD) to reuse relevant notes, prior runs, and task-specific constraints.
4. An evaluator that applies both deterministic keyword/format checks and an LLM-based judge for semantic correctness.

3.2 Core Methodological Foundations

This section presents the foundational methodological blocks that complement the enhanced architecture and support consistent implementation across evaluation tasks.

3.2.1 Model-Level Methodologies

1) LoRA/QLoRA Weight Adaptation: Parameter-efficient adaptation remains part of the baseline methodology for improving domain-specific behavior with lower compute overhead.

2) Reinforcement Learning Training: RL-based optimization remains relevant for improving action-selection behavior through feedback signals from environment interaction.

3) Multimodal Model Integration: Multimodal reasoning remains an optional extension for GUI-heavy workflows, while command-first execution is prioritized for lower latency.

3.2.2 Model Pool Used in This Work

1) github-copilot-gemini-3flash-preview: Fast general-purpose model for rapid drafting and short-horizon planning.

2) github-copilot-clause-sonnet-4.6: Balanced reasoning model for structured analysis and longer-form responses.

3) github-copilot-grok-code-fast-1: Fast code-oriented model for implementation and debugging loops.

4) github-copilot-gpt-5.3-codex: High-capability model for complex multi-step coding and reasoning tasks.

5) github-copilot-minmax-m2.5-free: Cost-aware model used for baseline and utility execution paths.

3.2.3 Prompt-Level Design

1) System Prompt Design: Defines behavior constraints, safety rules, and tool-usage policy.

2) User Prompt Interaction: Captures objective interpretation, task decomposition, and execution intent.

3.2.4 Harness-Level Architecture

1) Tool-Based Execution Framework: Connects model reasoning to file, shell, and app-level tools.

2) Environment Interaction Loop: Uses iterative observe-reason-act-update control for reliable completion.

3) Retrieval Utilities: Uses search-first retrieval and light indexing support to reduce redundant actions.

3.2.5 Context, Skill, and Memory Integration

1) Context Management: Maintains relevant state in long-horizon workflows and applies context compaction when needed.

2) Skill Abstraction: Encapsulates reusable procedures in tool/domain-specific skill modules.

3) Memory Systems: Retains prior decisions, constraints, and execution outcomes for future task reuse.

3.3 Key Methodologies

3.3.1 Tool-Augmented Agent Execution (Grounded Action Loop)

The core methodology is a structured interaction loop in which the agent:

1. Interprets the task specification from eval-config.json.
2. Plans a sequence of actions that produce required artifacts.

3. Routes each action to the correct tool/skill (for example, xlsx for spreadsheets, docx for Word redline operations, pdf for extraction and OCR-adjacent workflows, pptx for slide edits, and an image pipeline for EXIF sorting).
4. Executes tools inside an agent-browser environment so file outputs are real artifacts rather than purely text generation.
5. Performs retries or corrective actions when judge feedback indicates deviations.

Why this matters for evaluation: several partial outcomes in summary.json indicate execution and format gaps, such as malformed formulas, formatting deviations, and incorrect structured extraction. A tool-first approach reduces unverifiable output fabrication, but it still requires verification because tool outputs can fail silently or tools can be misused.

3.3.2 Retrieval-Augmented Planning with QMD (Memory + Constraint Reuse)

The proposed methodology adds a hybrid retrieval step before final execution:

1. Store task prompts, past run summaries, and tool instructions in QMD collections (for example, memory, sessions, and topics).
2. During planning, retrieve relevant constraints and examples of successful trajectories.
3. Inject retrieval results into the planning context to reduce missing constraints (for example, preserve font, or output separate labeled lines).

Why this matters: evaluation indicates that format constraints can be missed even when semantic intent is correct. Retrieval helps ensure the agent consistently follows formatting contracts.

3.3.3 Skill-Based Tooling (Office/Document Format Contracts)

The task suite requires reliable operation across multiple formats:

- xlsx**:** Formula audit, structured extraction, pivot-driven report generation, formula validation, and sheet-level sorting or totals.
- docx**:** Word redline and selection tasks, including multi-app report writing.
- pdf**:** Extraction from invoices and other PDF sources.
- pptx**:** Slide restructure and content insertion tasks.
- **Image pipeline:** EXIF-based sorting, deduplication, renaming, and image-table recognition.

Methodology:

1. Define format contracts for each skill, including required output schema, naming conventions, and acceptable deviations.
2. Expose tools that operate over real file artifacts rather than simulated values.
3. When tool outputs are incomplete, trigger corrective sub-skills (for example, auto-formatter CSV normalization).

3.3.4 Evaluator-Guided Self-Correction (Keyword + LLM Judge)

The current evaluation methodology uses:

- keywordScore: Deterministic scoring based on expected tokens, values, and format.
- judgeScore: An LLM judge that assigns PASS, PARTIAL, or FAIL with reasoning.

Proposed enhancement:

1. Convert judge reasoning into structured feedback categories (format violation, data fabrication risk, execution failure, safety and constraint compliance).
2. Route corrective action to the corresponding module (for example, rerun OCR with the correct tool, recalculate formulas, or apply template-driven formatting).

3.3.5 Better Verification for Evidence-Sensitive Tasks (Addressing Fabrication)

Current results include cases where the judge flags fabricated content, such as OCR extraction that appears invented without a proper OCR run, and office tasks where numbers appear simulated when source changes are missing.

Proposed verification upgrade (high-level):

1. Evidence binding: the agent must cite tool-produced evidence artifacts (for example, OCR output text, detected table bounding boxes, computed recalculation results, and diff summaries).
2. Post-processing checks:
    - OCR tasks: verify extraction provenance via recognized text segments and confidence thresholds when available.
    - Spreadsheet tasks: verify formula correctness by checking computed cells, not only saved formulas.
    - Docx redline tasks: verify underlying change XML or a redline diff artifact.

3.4 System Architecture (Textual)

At a high level, the runtime components are:

1. Task Interpreter: parses the task prompt, required artifacts, and acceptance criteria.
2. Planner/Router: generates a plan and selects tools or skills for each step.
3. Tool Executors:
    - Spreadsheet executor (xlsx skill)
    - Document executor (docx skill)
    - PDF executor (pdf skill)

- Presentation executor (pptx skill)
- Image pipeline (EXIF plus image table recognition)

4. Memory/Retrieval (QMD): retrieves prior trajectories and constraint examples.
5. Artifact Manager: tracks created and edited files plus intermediate outputs.
6. Output Verifier/Evaluator:
   - Runs keyword scoring
   - Runs LLM judge scoring and returns feedback

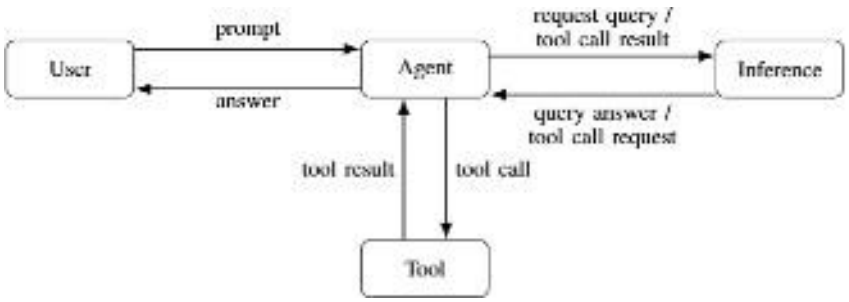

Figure 3.1: Agent-runtime interaction loop among user, agent, tool executor, and inference engine.

Analysis of Figure 3.1: The diagram captures a closed-loop control architecture in which the agent mediates between user intent, tool execution, and model inference. The user provides prompts to the agent, which then issues either inference requests (for reasoning and next-action generation) or tool calls (for grounded environment interaction). Tool results and inference responses are both fed back into the agent before forming a final user-facing answer. This structure is methodologically important because it separates planning from execution, enables iterative correction after intermediate failures, and supports verification-oriented workflows where evidence from tool outputs can be checked before response finalization.

3.5 Expected Benefits

1. **Reliability:** Reduces hallucination risk through real tool execution and evidence binding.
2. **Format compliance:** Improves adherence to strict output formatting contracts.
3. **Efficiency:** Lowers unnecessary trajectory steps via memory-guided planning and targeted retries.
4. **Generality:** Uses a consistent agent-browser loop with modular skills so new domains can be added.

3.6 Limitations of the Current Evaluation

1. Tool correctness is not guaranteed: tools can produce incorrect artifacts or be used improperly.
2. Evidence verification requires additional tooling: OCR, redline, and formula correctness often need deeper checks than file-exists validation.
3. Judge variability remains: LLM judging can produce inconsistent boundaries between PARTIAL and FAIL, though structured feedback routing helps mitigate this.

3.7 Improvement Roadmap for Future Iterations

The following additions are expected to provide the highest impact in subsequent versions:

1. Artifact-level evidence binding (OCR provenance, diff existence, recalculation validation).
2. Contract-based tool outputs (schema-validated structured outputs).
3. A multi-stage loop: Plan -> Execute -> Verify -> Fix instead of only Plan →Execute.

Chapter 4

Experimental Setup

4.1 Agent Architecture and Harness Design

The proposed agent architecture is structured as a layered system that separates user interaction, agent logic (harness), and model inference. The User Interface Layer captures initial instructions, which are processed by the Agent Logic Layer for intent interpretation and task decomposition. The Model Inference Layer performs reasoning and tool-selection policy execution. To optimize performance, the stack applies weight adaptation at the model layer, structured prompting at the prompt layer, and tool-based execution at the harness layer. This modular design supports consistent and safe behavior during complex operating-system tasks.

4.2 Model Configuration for Evaluation

In this study, these five Copilot-backed models are treated as competitor models and are evaluated under the same harness conditions. For in-between workflow execution, task routing is performed according to latency, reasoning depth, and coding complexity needs.

- **github-copilot-gemini-3flash-preview:** Used for quick decomposition and fast response turns.
- **github-copilot-clause-sonnet-4.6:** Used for analytical reasoning and structured technical explanations.
- **github-copilot-grok-code-fast-1:** Used for fast code generation, shell-command drafting, and repair loops.
- **github-copilot-gpt-5.3-codex:** Used for complex coding tasks, multi-step planning, and higher-accuracy edits.
- **github-copilot-minmax-m2.5-free:** Used as a cost-aware baseline for routine and utility tasks.

This competitor-model evaluation setup supports fair comparative analysis by assigning each stage of execution to an appropriate model profile while keeping workflow conditions consistent.

4.3 Execution Flow Across Layers

In the end-to-end execution path, user requests enter through the interface layer and are forwarded to the central harness in the Agent Logic Layer. The harness dispatches control and task-allocation signals to orchestration components, invokes tools and knowledge resources, and sends prompts to the inference layer. The inference layer then runs on a dedicated model service and returns outputs to the harness for action selection.

This layered view clarifies the separation between model execution, tool and knowledge access, orchestration policy, and serving infrastructure. The separation also improves maintainability, enables targeted optimization at each layer, and aligns with practical multi-layer agent architecture patterns used for general OS-task automation.

4.4 Evaluation

We evaluate the proposed methodologies across three practical task categories to measure reliability, efficiency, and robustness in real operating-system workflows.

4.4.1 Web Browsing

This subsection evaluates browser-centered tasks such as navigation, form interaction, information extraction, and multi-step web workflows. Each run records full trajectories (observations, tool calls, and final state checks) so we can measure task success rate, execution efficiency, and tool-usage validity under dynamic page conditions.

This configuration defines a hard web-browsing benchmark with 10 multi-step tasks across GitHub, Wikipedia, npm, arXiv, MDN, Hacker News, PyPI, Can I Use, and Stack Overflow. Each task requires real navigation plus targeted extraction, reasoning, and, in some cases, cross-page correlation rather than single-page lookup. All prompts enforce strict output for- matting with exactly four labeled lines, making responses straightforward to auto-validate. Most tasks also require saving a specific screenshot artifact, which provides proof of execution for the visited page. Scoring combines keyword and regex checks for structural and value validity, with LLM-based judging enabled for qualitative correctness. Overall, this bench- mark is designed to stress reliable browser-workflow execution under tighter complexity and longer timeouts.

4.4.2 File-management

This subsection focuses on filesystem operations including directory creation, file editing, metadata inspection, search, and structured project updates. We assess not only end-task completion but also action correctness, recovery after command errors, and redundant-step ratio to capture operational reliability in terminal-heavy scenarios.

This file-management evaluation suite defines nine practical document and data tasks spanning Excel, Word, OCR, file search, PowerPoint, and CSV cleanup. Each task includes a clear workflow prompt, a fixed timeout, and an estimated human baseline step count to benchmark agent effort. Several tasks require concrete output artifacts, such as audited spreadsheets, edited documents, normalized slides, scripts, and summary reports. Validation is strict: outputs must follow exact labeled response lines, pass regex and keyword checks, and often satisfy file-existence checks. Expected answer files are provided for many tasks,

and LLM-based judging is enabled across the suite for quality assessment. Overall, the suite serves as a structured benchmark for end-to-end file processing, transformation accuracy, and reliable output generation.

4.4.3 Google Workspace

This subsection evaluates productivity workflows across Google Workspace applications (for example, document updates, spreadsheet modifications, and cross-app task coordination). We track consistency of multi-step execution, correctness of state transitions, and the stabil- ity gains provided by constrained tools, adaptive memory, and context-management strate- gies.

This configuration defines eight Google Workspace evaluation tasks across Gmail, Calendar, and Drive using the gws CLI. The tasks use realistic workflow prompts, including triage, draft creation, conflict analysis, thread summary, drive audit, folder report, recurring-event audit, and sent-mail analytics. Outputs are tightly standardized with exact labeled lines so results remain machine-checkable and comparable across runs. Scoring combines keyword checks with regex-based validation for numeric and ID fields, while all tasks also enable LLM-based judging. Each task uses strict timeouts (approximately 10–12 minutes) and in- cludes estimated human baseline step counts for effort benchmarking. Several prompts also enforce run-unique artifacts (for example, timestamped draft or document names) to avoid collisions with previous executions.

Chapter 5

Results & Discussions

This chapter presents (1) per-task outcomes for the file-management-skills evaluation run (Gemini Flash example), (2) model-level comparisons across the evaluated agent models, and (3) qualitative discussion of dominant failure modes and their implications for improving LLM agents for general tasks.

5.1 Evaluation Scope and Key Definitions

- Suite under detailed analysis: file-management-skills (14 tasks).
- success_rate is computed as: passed_tasks / total_tasks * 100, where passed tasks correspond to status == "pass".
- Task status assignment uses score thresholds:
  - pass if score >= 80
  - partial if 30 <= score < 80
  - fail otherwise.

5.2 Model-Level Effectiveness (file-management-skills)

Table R3 summarizes effectiveness across five target models for the file-management-skills suite.

Table 5.1: Table R3: Model performance on file-management-skills

Model	Passed/Total	Success %	Mean Score	Mean Time (all, s)	Mean Time (pass, s)	Mean Steps	Mean Tool Calls	Thrash Ratio
github-copilot/claude-sonnet-4.6	10/14	71.4	86.9	214.98	205.40	13.64	16.36	0.07065
github-copilot/grok-code-fast-1	9/14	64.3	76.3	133.65	92.31	12.64	16.43	0.03511
github-copilot/gpt-5.3-codex	8/14	57.1	80.1	109.15	89.31	6.57	7.00	0.00893
github-copilot/gemini-3-flash-preview	7/14	50.0	80.2	65.34	66.65	8.71	9.57	0.07603
opencode/minimax-m2.5-free	6/14	42.9	62.8	297.85	143.34	8.43	12.45	0.03989

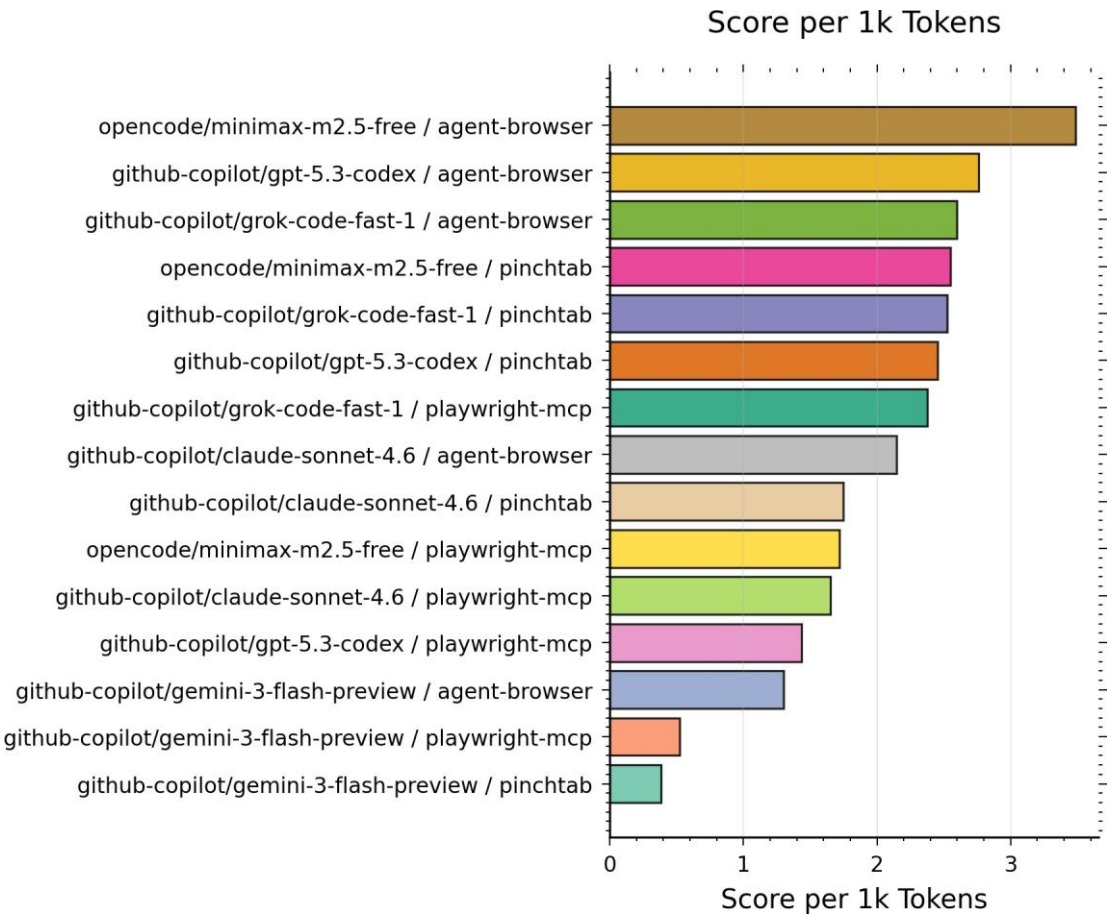

Figure 5.1: Score-per-token comparison across evaluated models in the analyzed suite.

5.2.1 Interpretation: Effectiveness vs Efficiency

- github-copilot/claude-sonnet-4.6 achieves the highest success rate (71.4%) and highest mean score (86.9), but with higher average wall-clock time.
- github-copilot/grok-code-fast-1 provides a stronger speed-performance balance among higher-success models.
- github-copilot/gpt-5.3-codex shows the most efficient execution profile (lowest mean tool calls and lowest thrash ratio), with moderate pass rate.
- github-copilot/gemini-3-flash-preview is fastest in mean elapsed time but has lower pass rate and higher thrash than GPT-5.3-codex.
- opencode/minimax-m2.5-free performs weakest on pass rate and mean score, with substantial time overhead.

5.3 Per-Task Results (Gemini Flash Example)

Tables R1 and R2 summarize per-task outcomes and category-level distribution for the Gemini Flash run.

Table 5.2: Table R1: Per-task outcomes (Gemini Flash example)

Task ID	Task Name	Format	Status	Score	Keyword	Judge	Tools	Time (s)	Thrash	Judge reason (short)
EVAL FM 001	Multi-Sheet Formula Audit & Summary	xlsx	pass	97	94	100	4	22.99	0.00	Correct results; only minor floating-point precision difference
EVAL FM 002	Tracked-Changes Redline Review & Selective Accept	docx	partial	75	100	50	15	74.63	0.14	Values mismatch source tracked-change state
EVAL FM 003	OCR Invoice Batch Extraction to Structured CSV	pdf	partial	75	100	50	15	54.81	0.00	Extraction judged as not grounded in actual OCR evidence
EVAL FM 004	Deep File Search with Metadata Filtering	–	partial	75	100	50	4	24.72	0.20	Structured output format deviation
EVAL FM 005	Slide Deck Restructure & Style Normalization	pptx	partial	75	100	50	7	46.69	0.00	Required font/style changes incomplete
EVAL FM 007	EXIF-Based Image Sort, Dedup, Rename	image	pass	100	100	100	10	50.05	0.18	Correct artifacts after script and file fixes
EVAL FM 008	Custom Tool Creation for CSV Normalization	xlsx	pass	90	80	100	6	31.19	0.00	Tool created and executed; normalized CSV output
EVAL FM 009	Extract Failed Students from Exam Sheet	xlsx	partial	40	80	0	7	80.08	0.29	Corrupt rows included from legend/footnote regions
EVAL FM 010	Cross-Format Pivot-to-Report Generation	docx,xlsx	pass	90	80	100	7	35.27	0.00	Pivot and report generation complete
EVAL FM 011	Multi-App Financial Analysis and Summary	docx,pdf,xl	xpass	100	100	100	8	76.33	0.00	Outputs satisfy content and structure requirements
EVAL FM 012	Cell-Level Formula Validation Task	xlsx	partial	63	75	50	5	30.90	0.00	Malformed formulas produced null computed values
EVAL FM 013	Sheet-Level Merge/Sort/Total Transformation	xlsx	partial	63	75	50	14	136.38	0.14	Sorting and totals were incorrect
EVAL FM 014	Image Table Recognition to Spreadsheet	xlsx	pass	90	80	100	10	118.07	0.00	Correct table extraction and spreadsheet output
EVAL FM 015	Agenda Slide Insertion After Title Slide	pptx	pass	90	79	100	22	132.64	0.11	Slide added at expected position with required content

Description: Figure 5.2 shows that the dominant non-success category across domains is execution, while comprehension, resource, and navigation errors remain compar- atively less frequent. This indicates that most failures occur after intent understanding, during action realization and tool-operation stages. The file-management-skills and google-workspace-gws-cli domains show particularly strong execution-error presence, suggesting that verification-oriented control loops and stricter tool-output validation are likely to produce the largest reliability gains.

5.4 Qualitative Discussion: Dominant Failure Modes

5.4.1 Evidence-sensitive tasks

Evidence-sensitive tasks remain vulnerable to fabrication-like outcomes when outputs are not tightly bound to tool-produced artifacts. The tracked-change and OCR extraction tasks illustrate this pattern: formatting may appear valid, while semantic grounding fails.

5.4.2 Formatting contracts and style constraints

Strict formatting and style constraints frequently drive partial outcomes even when core task logic is mostly correct. This suggests that template-constrained post-processing should be treated as a mandatory verification stage.

Table 5.3: Table R2: Pass/partial distribution by document type (Gemini Flash)

Document type/domain	Pass	Partial	Fail	Observed notes
Spreadsheets (xlsx)	5	3	0	Strong parsing/pivot/report generation; weaker formula integrity and sorting/total constraints
Word (docx)	2	1	0	Partial linked to tracked-change evidence mismatch
PDF (pdf)	1	1	0	Partial linked to extraction-grounding issues
Presentations (pptx)	1	1	0	Slide structure edits work; style constraints are brittle
Images/EXIF pipeline	1	0	0	End-to-end tool execution and artifact validity observed

5.4.3 Spreadsheet verification requirements

Spreadsheet reliability depends on both formula syntax and computed-value correctness. Formula-writing alone is insufficient; recalculation and invariant checks are necessary to prevent silent partial failures.

5.5 Efficiency, Thrashing, Safety, and Constraints (Gemini Flash)

Run-level aggregates indicate: meanSteps = 8.71, meanStepsSuccessful = 9, meanSafetyScore = 100, harmfulAttemptCount = 0, constraintViolationCount = 4, and meanThrashRatio = 0.08. Error-category counts are split between none = 7 and execution = 7.

These outcomes indicate that strong safety behavior does not automatically guarantee artifact-level correctness. Reliability improvements should therefore prioritize evidence bind- ing and contract-level verification.

5.6 Cross-Suite Comparison (Best Models by Success Rate)

5.7 Recommendations for Methodological Improvement

1. **Evidence binding:** Verify extraction, redline, and formula outputs against tool-generated artifacts.
2. **Contract-based normalization:** Enforce schema and formatting constraints via deterministic post-processing.

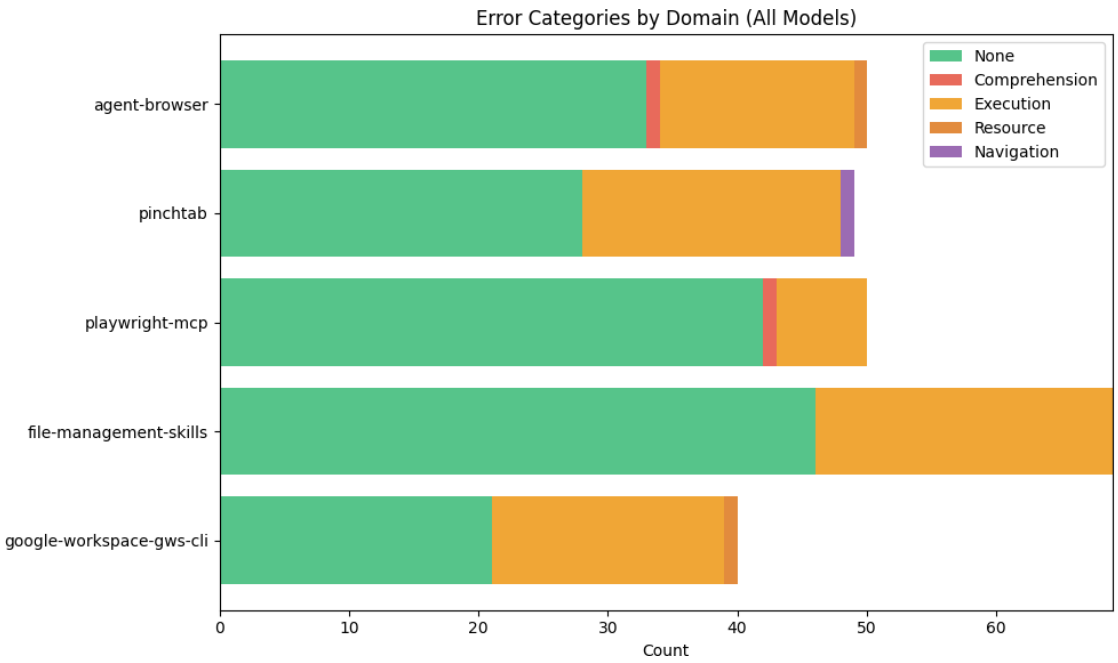

Figure 5.2: Error-category distribution by domain aggregated across evaluated models.

3. **Verification loop:** Use Plan →Execute →Verify →Fix rather than execution-only flow.

4. **Spreadsheet invariants:** After formula insertion, verify computed cells, sorting, and total constraints.

Table 5.4: Table R4: Best model per suite by success rate

Suite	Best model(s)	Best success %	Evidence summary
file-management	github-copilot/gemini-3-flash-preview; github-copilot/grok-code-fast-1	87.5	Highest observed success rate among evaluated models
file-management-skills	github-copilot/claude-sonnet-4.6	71.4	Highest observed success rate among evaluated models
agent-browser	github-copilot/gpt-5.3-codex	80.0	Highest observed success rate among evaluated models
pinchtab	github-copilot/claude-sonnet-4.6	70.0	Highest observed success rate among evaluated models
playwright-mcp	github-copilot/gemini-3-flash-preview; github-copilot/gpt-5.3-codex; opencode/minimax-m2.5-free	90.0	Joint highest observed success rate
google-workspace-gws-cli	github-copilot/gemini-3-flash-preview; github-copilot/gpt-5.3-codex; github-copilot/grok-code-fast-1; opencode/minimax-m2.5-free	50.0	Joint highest observed success rate

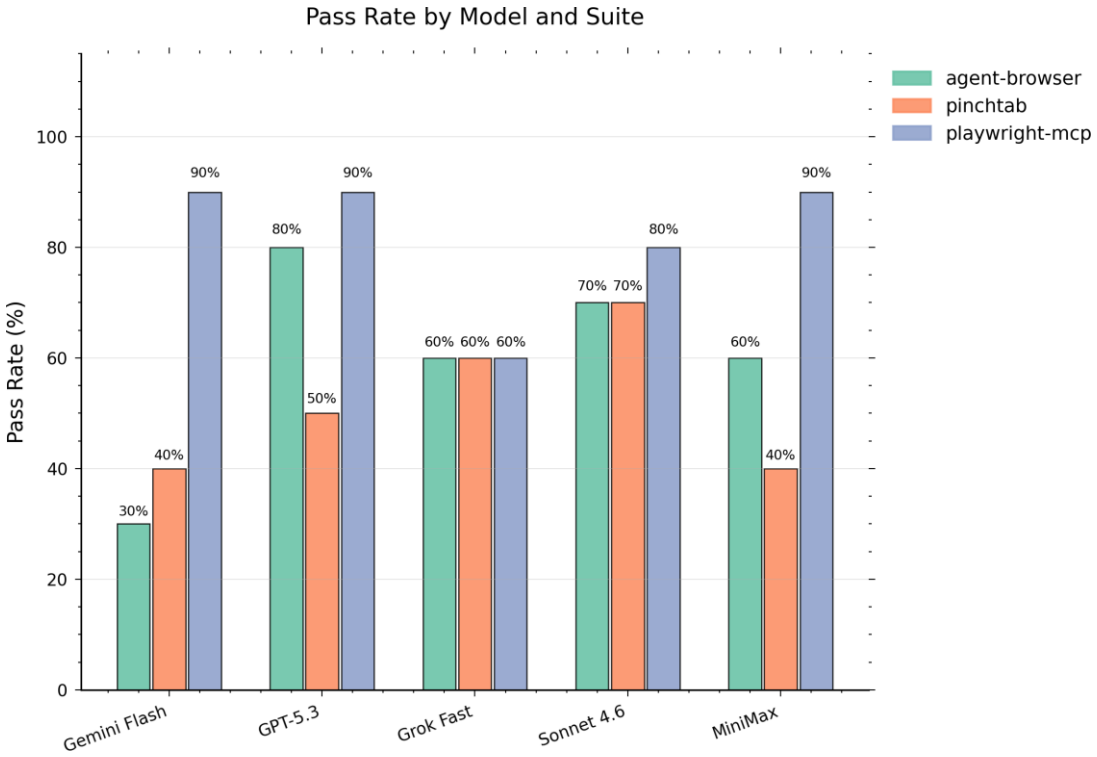

Figure 5.3: Suite-wise pass-rate comparison across evaluated models.

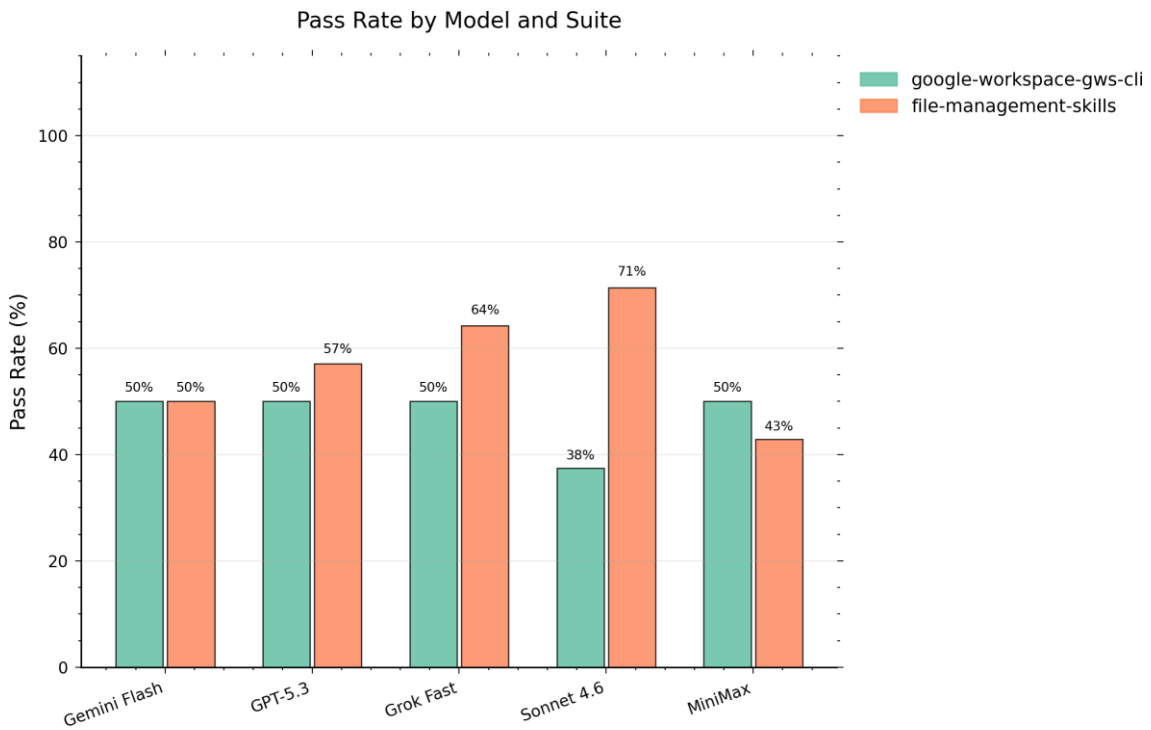

Figure 5.4: Model-wise pass-rate comparison across two key suites: file-management-skills and google-workspace-gws-cli.

Chapter 6

Conclusion & Future Work

6.1 Conclusion (What the Evaluation Shows)

This project evaluates tool-augmented LLM agents for end-to-end general tasks across multiple document formats (Excel, Word, PDF, PowerPoint) and an image pipeline, using the established evaluation framework with deterministic keyword scoring and LLM-judge rubrics.

6.1.1 Effectiveness is strongly model-dependent (file-management-skills, 14 tasks)

The comparative results show clear model-dependent performance differences in the file-management-skills suite:

- github-copilot/claude-sonnet-4.6 achieves the best success rate at **71.4%** (10/14 passes).
- github-copilot/grok-code-fast-1 follows at **64.3%** (9/14).
- github-copilot/gpt-5.3-codex reaches **57.1%** (8/14), with notably efficient tool usage and very low thrashing.
- github-copilot/gemini-3-flash-preview reaches **50.0%** (7/14) and is fast, but more frequently exhibits verification and formatting drift.
- opencode/minimax-m2.5-free is lowest at **42.9%** (6/14) and also shows greater time overhead in aggregate behavior.

6.1.2 The dominant failure driver is execution and verification, not safety

Detailed outcomes indicate that the primary reliability bottleneck is verification-grade correctness rather than harmful behavior:

- Evidence-sensitive tasks (OCR-like extraction and tracked-change verification) can produce fabrication-like outputs when extracted values are not tied to artifact evidence.

- Structured outputs remain brittle: small formatting contract violations can convert otherwise correct intent into partial outcomes.
- Spreadsheet failures commonly arise from formula structure and downstream recalculation/sorting/aggregation invariants, not from prompt comprehension alone.

Safety metrics remain strong in the analyzed run (meanSafetyScore = 100, harmfulAttemptCount = 0), reinforcing that the major gap is correctness validation under execution constraints.

6.2 Key Takeaways (Methodological Findings)

1. A two-stage evaluation strategy (deterministic checks plus LLM-judge rubrics) effectively separates artifact presence from artifact correctness.
2. Reliability improvements should prioritize verification-grade constraints, especially:
    - evidence binding for extraction and redline-sensitive tasks,
    - strict contract normalization for formatting and style,
    - post-execution spreadsheet verification using recalculation and invariant checks.
3. Efficiency metrics (tool calls, thrashing, trajectory length) show that some models can achieve competitive scores with fewer corrective behaviors, indicating stronger control-loop stability.

6.3 Future Work (Concrete Next Steps)

6.3.1 Evidence-Binding Verification (OCR, Redline, and Table Extraction)

Introduce explicit verifiers that require proof-of-extraction from tool-produced artifacts before accepting task outputs.

- OCR and table extraction: require provenance indicators such as recognized text-cell matching or structured bounding evidence.
- Tracked-change workflows: verify that acceptance/rejection operations and final document diffs satisfy rubric constraints.

6.3.2 Contract-Based Tool Outputs (Schema + Repair)

Adopt schema-validated intermediate artifacts instead of free-form output generation.

- Require structured JSON or typed intermediate outputs for extracted fields.
- Trigger automatic repair steps when schema checks fail.

6.3.3 Multi-Stage Plan -> Execute -> Verify -> Fix Loop

Use verifier feedback to route corrective actions only to failed components, reducing unnecessary trajectory drift.

- Execute tools to produce required artifacts.
- Verify outputs using evidence and contract checkers.
- Repair only failing components and re-validate.

6.3.4 Ablation Studies Across Methodological Components

Run controlled ablations to quantify contribution of each methodological upgrade:

- with vs without evidence binding,
- with vs without contract normalization,
- with vs without verification loops,
- with vs without memory and retrieval guidance.

6.3.5 Evaluation Expansion

Expand benchmarking diversity to improve robustness estimates:

- additional templates per document type,
- stronger edge cases for formula, sorting, and totals invariants,
- OCR and redline variants with increased layout noise,
- repeated-run evaluation for variance and stability analysis.

6.4 Final Summary

The evaluation demonstrates that general-task, tool-augmented LLM agents can complete many artifact-generation workflows reliably, but the strongest reliability lever is verification-grade correctness. Evidence binding combined with contract-based post-execution validation is central to reducing partial outcomes. A structured Plan -> Execute -> Verify -> Fix workflow is expected to convert a substantial portion of current partial outcomes into pass by addressing dominant evidence and formatting drift failure modes.

Appendix

Appendix A: Extended Evaluation Notes

This appendix provides detailed supporting material for the main chapters, including expanded evaluation criteria, execution-check templates, error-taxonomy guidance, and reproducibility checklists used to keep the study process consistent across task categories. The intent is to make the report operationally reproducible and methodologically transparent without overloading the core narrative chapters.

The appendix is organized as follows. Appendix A summarizes evaluation assumptions and run discipline. Appendix B provides extended rubric guidance for deterministic checks and judge-assisted scoring. Appendix C presents a practical error-triage framework used when classifying partial outcomes. Appendix D gives additional reviewer checklists for artifact validation. Appendix E provides an ablation planning template for future experiments.

Appendix B: Extended Rubric Guidance

B.1 Deterministic Checks

Deterministic checks are used to ensure output structure, required fields, and key-value correctness. In practice, these checks are effective at identifying formatting defects, missing labels, malformed records, invalid value types, and obvious arithmetic mismatches. However, deterministic checks cannot fully validate whether values were extracted from true artifact evidence.

For this reason, deterministic checks are applied as a first validation gate and are interpreted as necessary but not sufficient for final pass qualification. A task can satisfy format-level constraints while still failing semantic integrity under judge review.

B.2 Judge-Assisted Semantic Checks

Judge-assisted checks evaluate semantic faithfulness and evidence plausibility. The judge is expected to identify whether reported outputs are grounded in actual artifact state, whether mandatory transformation constraints were applied, and whether the response aligns with task intent beyond superficial formatting.

To reduce inconsistency across judge outcomes, the review prompt should explicitly separate the following criteria: (1) artifact grounding, (2) transformation correctness, (3) format compliance, and (4) task completion scope. This separation improves interpretability of partial outcomes and helps map failure causes to specific repair actions.

B.3 Recommended Decision Hierarchy

1. Verify artifact existence and expected file outputs.
2. Verify deterministic schema/format and required-value constraints.
3. Verify semantic grounding and task-intent fulfillment under judge review.
4. If disagreement exists between deterministic and judge checks, prioritize evidence-grounded correctness and classify as partial until verified.

Appendix C: Error Taxonomy and Triage Procedure

The error taxonomy used in the report includes the categories none, comprehension, execution, resource, and navigation. To improve consistency during post-run analysis, the following triage procedure is recommended.

C.1 Triage Steps

1. Identify the first observable divergence from the required workflow.
2. Determine whether divergence originated from intent interpretation, action selection, tool execution, resource availability, or navigation state.
3. Record evidence excerpts from logs/artifacts supporting the assigned category.
4. Assign secondary tags where needed (for example: formatting drift, extraction mis-match, formula integrity failure, or retry-thrashing).
5. Recommend a targeted repair action for each failed or partial task.

C.2 Category Interpretation Notes

- **Comprehension:** Incorrect interpretation of objective or constraints before major execution begins.
- **Execution:** Action plan is reasonable, but implementation fails due to tool misuse, incorrect command arguments, transformation defects, or missing verification.
- **Resource:** Failures linked to unavailable dependencies, inaccessible files, permission limits, or runtime constraints.
- **Navigation:** Failures associated with incorrect page/state traversal or broken context progression in interactive environments.

In this report, execution-origin errors are most dominant, indicating that improvements should prioritize post-execution verification, contract-normalization checks, and evidence-binding mechanisms.

Appendix D: Artifact Validation Checklists

D.1 Spreadsheet Tasks

1. Confirm required sheets are present and correctly named.
2. Validate formula syntax for target cells and ranges.
3. Recalculate workbook and validate computed values against expected invariants.
4. Verify sort order and aggregation totals after transformation.
5. Confirm output naming and storage path requirements.

D.2 Document Redline Tasks

1. Verify track-change operations exist in the target document state.
2. Confirm acceptance/rejection actions match task constraints.
3. Validate summary counts against actual redline content.
4. Ensure final exported artifact preserves required formatting.

D.3 OCR and Extraction Tasks

1. Confirm extraction was performed on the correct input artifact.
2. Validate extracted fields against source text regions.
3. Check numeric fields for format and range consistency.
4. Record provenance notes where extraction confidence is ambiguous.

D.4 Presentation Tasks

1. Validate slide insertion index and sequence placement.
2. Validate content completeness in required placeholders.
3. Check style constraints (font, size, alignment, theme elements).
4. Confirm exported deck naming and versioning requirements.

Appendix E: Reproducibility Checklist

The following checklist is recommended for every major run to ensure reproducibility and fair cross-model comparison.

Item	Required Record
Task configuration	Exact configuration identifier and version/hash used for the run.
Model roster	Full evaluated model list and ordering for the run.
Runtime environment	OS, browser/runtime versions, and key execution settings.
Scoring pipeline	Deterministic check definition, judge settings, and thresholds.
Artifacts	Output files, logs, and any supporting evidence snapshots.
Failure annotations	Error category assignment and repair recommendation notes.
Aggregation method	Formulae for success rate, means, and summary statistics.

Appendix F: Ablation Plan Template

Future iterations should isolate the impact of each reliability mechanism using controlled ablation groups under matched task conditions.

Ablation Factor	Variant Comparison	Primary Outcome Measures
Evidence binding	Enabled vs disabled	Judge score stability, fabrication-like partial rate, extraction fidelity.
Contract normalization	Enabled vs disabled	Formatting-compliance pass ratio, deterministic check failure reduction.
Verification loop	Plan-Execute vs Plan-Execute-Verify-Fix	Partial-to-pass conversion rate, retry efficiency, thrash ratio.
Memory/retrieval support	Enabled vs disabled	Constraint adherence consistency, long-horizon completion stability.

Recommended reporting format for each ablation: include task-level deltas, confidence ranges over repeated runs, and explicit examples of corrected failure modes.

Appendix G: Additional Discussion Notes

A recurring observation across evaluations is that many partial outcomes are not due to inability to produce an artifact, but due to incorrect or weakly verified artifact content. This distinction is important for designing reliable improvement strategies. In practical terms,

improving generation capability alone is insufficient; strong verification discipline is required for artifact-centric tasks.

Another practical observation is that low thrash ratio does not always imply high semantic correctness. Efficient trajectories can still produce confidently incorrect outputs when verification checkpoints are weak. Therefore, efficiency metrics should be interpreted jointly with evidence-grounding indicators and judge rationale.

Finally, model comparisons should be interpreted at the suite level rather than using a single global ranking. Different domains stress different capabilities: some suites emphasize tool-execution robustness, others emphasize constraint-heavy formatting, and others emphasize cross-application state consistency. A robust future system should optimize for balanced cross-suite reliability rather than isolated peak performance in one category.

Bibliography

[1] Alessandro Armando, Gabriele Costa, Alessio Merlo, *Bring Your Own Device, Securely*, Proceedings of the 28th Annual ACM Symposium on Applied Computing, ACM, New York, USA, pp. 1852–1858, 2013.

[2] Rebecca Copeland, Noel Crespi, *Analyzing Consumerization: Should Enterprise Business Context Determine Session Policy?*, Proceedings of the 16th International Conference on Intelligence in Next Generation Networks (ICIN), IEEE, pp. 187–193, 2012.

[3] Prashant Kumar Gajar, Arnab Ghosh, Shashikant Rai, *Bring Your Own Device (BYOD): Security Risks and Mitigating Strategies*, Journal of Global Research in Computer Science, Vol. 4, No. 4, April 2013.

[4] Sean Chung, Sam Chung, Teresa Escrig, Yan Bai, Barbara Endicott-Popovsky, *2TAC: Distributed Access Control Architecture for "Bring Your Own Device" Security*, Proceedings of the International Conference on Biomedical Computing, 2012.

[5] Dennis Titze, Philipp Stephanow, Julian Schutte, *A Configurable and Extensible Security Service Architecture for Smartphones*, Proceedings of the 27th International Conference on Advanced Information Networking and Applica- tions Workshops, IEEE, 2013.

[6] Geon Il Heo, Yong Jun Park, Won Hyung Park, *Vulnerability of Information Disclosure in Data Transfer Section for Constructing a Safe Smart Work Infrastructure*, Multimedia Tools and Applications, Vol. 74, No. 20, pp. 8831–8847, 2015.

[7] Dennis Gessner, Joao Girao, Ghassan Karame, Wenting Li, *Towards a User-Friendly Security-Enhancing BYOD Solution*, NEC Technical Journal, Vol. 7, No. 3, pp. 113–116, 2013.

[8] Y. S. Hanay, T. Wolf, *Techniques for Policy Enforcement on Encrypted Network Traffic*, MILCOM 2009 – IEEE Military Communications Conference, pp. 1–7, 2009.

[9] Ioannis Koskosas, *A Short Literature Review in Information Systems Security Management Approaches*, Business Excellence and Management, Vol. 3, No. 2, pp. 5–15, 2013.

[10] Antonio Scarfò, *New Security Perspectives Around BYOD*, Seventh International Conference on Broadband, Wireless Computing, Communication and Applications, IEEE, pp. 446–451, 2012.

[11] Mohammad Nauman, Sohail Khan, Xinwen Zhang, *Apex: Extending Android Permission Model and Enforcement with User-defined Runtime Constraints*, Proceedings of the 5th ACM Symposium on Information, Computer and Communications Security (ASIACCS), pp. 328–332, 2010.

[12] Ionut Andronache, Claudiu Nisipasiu, *Web Single Sign-On Implementation: SimpleSAMLphp Application*, Journal of Mobile, Embedded and Distributed Systems, Vol. 3, No. 1, 2011.

Acknowledgement

We wish to express our sincere gratitude to **Dr. Sanjay U. Bokade, Principal** and **Prof. S. P. Khachane , H.O.D.** of Department Computer Engineering of Rajiv Gandhi Institute of Technology for providing us an opportunity to do our project work on **"Evaluation of Methodologies for Improving and Enhancing LLM Agents for General OS Tasks"**.

This project bears on imprint of many peoples. We sincerely thank our project guide **Prof. Savita Lade** for his/her guidance and encouragement in carrying out this project report work.

Finally, we would like to thank our colleagues and friends who helped us in completing project work successfully.

1. Aditya Ahire
2. Aniruddha Chaudhari
3. Raj Malave
4. Soham Mahale